Claude Certified Architect Foundation: Simulators and Labs

TGS-2026061312 · Six simulators, one per official exam scenario · 61 labs by topic, drawn from the practice exam in the Tertiary Markdown Vault

How this folder works

A simulator is a website, served from your laptop, that runs one working system. A lab is one exam situation you run on a simulator. Each topic has its own set of labs, taught straight after the concept they apply. The exam draws 4 scenarios from a bank of 6, so you practise on all six simulators.

Simulator	Official exam scenario	Primary domains	Runs on	Folder
1	Customer Support Resolution Agent	1, 2, 5	Agent SDK	`simulators/simulator-1-customer-support/`
2	Code Generation with Claude Code	3, 5	Agent SDK	`simulators/simulator-2-code-generation/`
3	Multi-Agent Research System	1, 2, 5	Agent SDK	`simulators/simulator-3-multi-agent-research/`
4	Developer Productivity with Claude	2, 3, 1	Agent SDK	`simulators/simulator-4-developer-productivity/`
5	Claude Code for Continuous Integration	3, 4	Agent SDK	`simulators/simulator-5-ci-cd/`
6	Structured Data Extraction	4, 5	Agent SDK	`simulators/simulator-6-structured-extraction/`

Where the labs come from

- 40 labs: all 40 questions of the SkillCertPro practice exam Part 1, from a third-party source. These are unofficial third-party questions, reconstructed from a video transcript. They are practice material, not real exam items.
- 7 labs: official sample questions from the exam guide.
- 14 labs: built directly from task statements the practice exam does not reach, and from Scenario 4, which the video does not cover.

What you need

- `uv` (https://docs.astral.sh/uv/) and a Claude login. That is all: uv fetches Python and the Claude Agent SDK.
- Pick the model on the page, or with `LAB_MODEL` in a terminal. On `claude-sonnet-5-5`, the default, Lab 1 costs one to three US cents and the long research run in Lab 3 about 40 cents. `claude-haiku-5-5` spends least.

© 2026 Tertiary Infotech Academy · UEN 201200696W

Open the simulators

```
cd labs
uv sync                # first time only
claude auth login      # first time only
uv run serve.py        # opens http://localhost:8765, or the next free port
```

Six pages, one per simulator, all in Python on the Claude Agent SDK, through your own Claude login. No API key. Each page has Start here (the scenario and steps), Play it live, and its own labs. In a lab, you pick an answer, see at once why it is right or wrong, then watch the right answer work, with a built-in terminal on the page. Lab numbers follow the exam domains, so each simulator holds its own set. A live run uses your Claude allowance.

Prefer Claude Code? Open it in the `labs` folder and type `/lab 9`. `/simulators` starts the website.

Prefer a terminal? From a simulator's folder, run the command under any lab, such as `uv run support_agent.py --case CS-01`. `uv run ../../run_lab.py 9` walks you through a lab.

All labs

An asterisk marks a lab run in class. The rest are self-practice.

© 2026 Tertiary Infotech Academy · UEN 201200696W

Topic 1: Agentic Architecture & Orchestration

Lab	Task	Source	What it proves	Command
1*	1.1	Task statement	Watch one refund go round the loop	`uv run support_agent.py --case CS-01`
2	1.1	Practice Q17	Claude re-asks because the tool_result never came back	`uv run casekit.py --case UC-17`
3*	1.2	Guide Q7	Every subagent succeeds, and the report still misses half the topic	`uv run research_system.py`
4	1.2	Practice Q36	Same brief to every subagent, same findings four times	`uv run extract.py --case UC-36`
5*	1.3	Practice Q25	A /parallel-review that can't fan out because Task isn't allowed	`uv run casekit.py --case UC-25`
6*	1.3	Practice Q2	Same reviewers, same findings, and the PR comment loses them all	`uv run ci_subagents.py --pass-findings`
7	1.3	Practice Q10	Polished report, weak citations: route information, not just tasks	`uv run research_system.py --case UC-10`
8	1.3	Practice Q35	Later subagents re-run searches the first ones already finished	`uv run research_system.py --case UC-35`
9*	1.4	Guide Q1	12% of cases skip get_customer: fix it with a gate, not a sentence	`uv run support_agent.py --case CS-03`
10*	1.4	Practice Q3	Approval must never post before the passing test artifact exists	`uv run hook_gate.py`
11	1.4	Practice Q12	Claude guesses a field and submits without validating	`uv run extract.py --case UC-12`
12*	1.5	Task statement	The prompt forgot the $500 rule. The hook did not.	`uv run support_agent.py --case CS-02 --prompt prompts/system-noguard.md --tag noguard`
13	1.5	Practice Q9	Shrink noisy npm test output before Claude reads it	`uv run casekit.py --case UC-9`
14	1.5	Task statement	rm -rf meets a hook, not a polite request	`uv run explorer.py --case UC-D6`
15*	1.6	Practice Q31	60-page packets: section passes first, reconcile second	`uv run extract.py --case UC-31`
16*	1.7	Practice Q5	Two attempts, one analysis: fork the session, checkpoint the files	`uv run casekit.py --case UC-5`
17	1.7	Task statement	Resume, and say exactly which file changed	`uv run explorer.py --case UC-D4`

© 2026 Tertiary Infotech Academy · UEN 201200696W

Topic 2: Tool Design & MCP Integration

Lab	Task	Source	What it proves	Command
18*	2.1	Practice Q11	One analyze_document tool does three jobs badly	`uv run extract.py --case UC-11`
19*	2.2	Practice Q29	The refund failed. The customer was told it worked.	`uv run support_agent.py --case CS-11`
20	2.2	Task statement	Retry the timeout once, then answer from the real record	`uv run support_agent.py --case CS-06`
21*	2.3	Practice Q15	Review and test generation fight over one Claude call	`uv run casekit.py --case UC-15`
22	2.3	Guide Q9	Give synthesis one narrow tool, not the whole web	`uv run research_system.py`
23*	2.4	Task statement	Commit the server, never the token	`uv run simulator.py 5`
24	2.4	Task statement	Read the catalogue before opening a single file	`uv run explorer.py --case UC-D2`
25*	2.5	Practice Q37	/find-callers needs Grep with a glob, not Bash	`uv run explorer.py --case UC-37`
26	2.5	Task statement	Edit says the anchor isn't unique. Now what?	`uv run explorer.py --case UC-D1`

© 2026 Tertiary Infotech Academy · UEN 201200696W

Topic 3: Claude Code Configuration & Workflows

Lab	Task	Source	What it proves	Command
27*	3.1	Task statement	Which instructions actually reached Claude?	`uv run simulator.py 1`
28	3.2	Guide Q4	A /review command the whole team gets through git	`uv run simulator.py 2`
29*	3.2	Practice Q16	Keep a noisy case review out of your main session	`uv run case_review.py CS-02`
30*	3.3	Practice Q7	Testing rules for test files spread across every folder	`uv run rules_sim.py D`
31*	3.4	Practice Q8	Replacing the auth layer: nobody agrees on the boundary, so plan	`uv run casekit.py --case UC-8`
32	3.4	Practice Q30	Plan first, then approve into an editing mode	`uv run casekit.py --case UC-30`
33*	3.4	Practice Q34	One guard clause doesn't need a plan	`uv run casekit.py --case UC-34`
34	3.5	Practice Q14	Unknown requirements? Let Claude interview you first	`uv run casekit.py --case UC-14`
35	3.5	Practice Q27	Two linked defects go in one detailed request	`uv run research_system.py --case UC-27`
36*	3.6	Practice Q18	The nightly job hangs because nobody can press Enter	`uv run casekit.py --case UC-18`
37	3.6	Practice Q38	A shared slash command still needs -p in CI	`uv run casekit.py --case UC-38`

© 2026 Tertiary Infotech Academy · UEN 201200696W

Topic 4: Prompt Engineering & Structured Output

Lab	Task	Source	What it proves	Command
38*	4.1	Practice Q39	"Only high confidence" didn't quiet the reviewer	`uv run casekit.py --case UC-39`
39	4.1	Practice Q20	Switch off the one noisy category, keep the accurate ones	`uv run research_system.py --case UC-20`
40*	4.2	Practice Q23	"Clean this up" means three things: show which one you mean	`uv run casekit.py --case UC-23`
41	4.3	Practice Q28	Vague passages forced into a category they don't fit	`uv run research_system.py --case UC-28`
42	4.3	Task statement	Know your tool_choice options, and check what came back	`uv run extract.py --tool-choice forced --max-docs 2`
43*	4.4	Practice Q4	Valid JSON, wrong refund: add business rules	`uv run support_agent.py --case CS-13`
44*	4.4	Task statement	Retry when the format is wrong, stop when the data is absent	`uv run extract.py`
45*	4.5	Practice Q1	Prove the template before 35,000 transcripts	`uv run triage.py --pilot`
46	4.5	Practice Q26	Client tools can't run inside a batch	`uv run triage.py --batch`
47	4.5	Practice Q32	Nightly job on Batches, upload flow stays synchronous	`uv run extract.py --case UC-32`
48*	4.6	Practice Q6	The author reviewing its own draft misses its own errors	`uv run research_system.py --case UC-06`
49	4.6	Practice Q19	Source-level passes first, integration second	`uv run research_system.py --case UC-19`
50*	4.6	Guide Q12	14 files, shallow comments: split the review	`uv run ci_review.py; echo "exit $?"`

© 2026 Tertiary Infotech Academy · UEN 201200696W

Topic 5: Context Management & Reliability

Lab	Task	Source	What it proves	Command
51*	5.1	Practice Q40	Refund amount jumped to the wrong order	`uv run support_agent.py --case CS-12`
52	5.1	Practice Q13	40-field tool results push the review to its limit	`uv run casekit.py --case UC-13`
53	5.1	Practice Q21	Taxonomy drift: write it to a state file	`uv run research_system.py --case UC-21`
54*	5.2	Guide Q3	55% first-contact resolution: calibrate with criteria and examples	`uv run support_agent.py --case CS-05 --case CS-08 --case CS-09 --case CS-10`
55*	5.3	Guide Q8	The search timed out: tell the coordinator everything	`uv run research_system.py`
56*	5.4	Task statement	Delegate the noisy search, keep a scratchpad	`uv run explorer.py --case UC-D3`
57	5.4	Practice Q22	Keep discovery out of the main thread	`uv run extract.py --case UC-22`
58	5.4	Task statement	Crash recovery from a manifest	`uv run explorer.py --case UC-D5`
59	5.5	Practice Q33	Acceptance looks great. Two fields don't.	`uv run casekit.py --case UC-33`
60*	5.5	Task statement	The aggregate masks a segment: break it and watch	`uv run extract.py`
61*	5.6	Practice Q24	A vendor benchmark isn't a peer-reviewed study	`uv run research_system.py --case UC-24`

Ground rules

- Every simulator makes real Claude calls through the Agent SDK, against mock backends and fixtures. Nothing touches production, and nothing here needs a production credential.
- Change a fixture, rerun, and watch the evidence. Each lab's PDF names one thing to change.
- Simulator 6 and the batch labs run the batch workflow locally: same custom_id steps, but no 50% discount or 24-hour window.

© 2026 Tertiary Infotech Academy · UEN 201200696W